Сем9

Сегодня 21:23 Без категории

Стек: LIFO (Last In, First Out) структура данных.

В данном курсе под стеком мы будем подразумевать аппаратный стек.

Аппаратный стек - область памяти, которую выделяет нам операционная система при запуске программы. Стек используется для хранения временных данных, адресов возврата из функций, сохраняемых регистров, локальных переменных, параметров вызова функций.

- ESP - регистр, который указывает на вершину стека.
- Стек растет вниз, т.е. память растущая по адресу до ESP

считается свободной, после-
занятой.

- push – положить в стек
  pop – достать из стека

- push eax $\Leftrightarrow$ $\begin{cases} ESP = ESP - 4 \\ [ESP] = EAX \end{cases}$

- pop eax $\Leftrightarrow$ $\begin{cases} EAX = [ESP] \\ ESP = ESP + 4 \end{cases}$

[stack_simple.s]

Для чего используется стек:

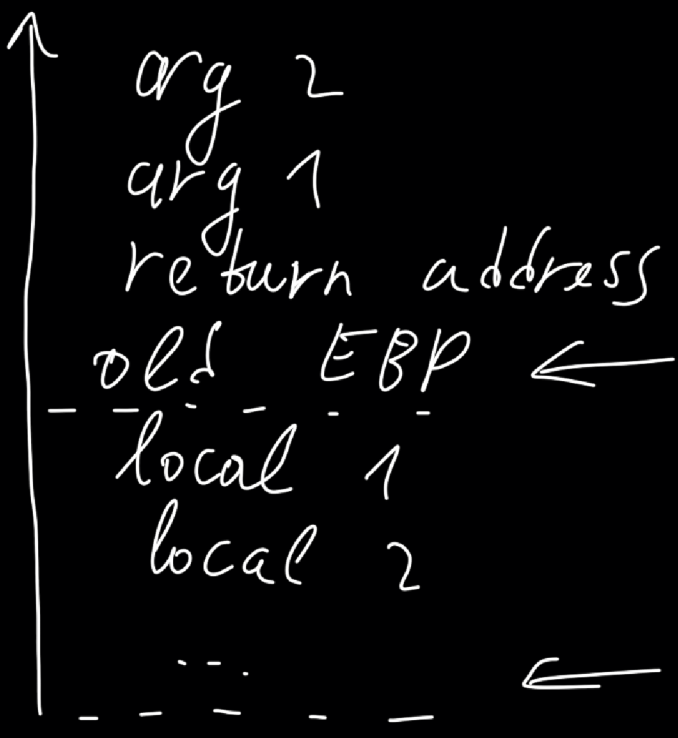
1) Для сохранения регистров:
   - регистры, которые должна сохранить вызываемая функция: caller-saved
   - регистры, которые сохраняет вызываемая функция: callee-saved

2) Адресов возврата из функции
   [call_ret.s]

3) Параметров вызова функций:

4) Локальных переменных функции

• Стековый фрейм:
Это часть стека которая выделяется для каждого вызова функции.
Он содержит:
1) аргументы функции
2) адрес возврата
3) адрес предыдущего фрейма
4) локальные переменные!



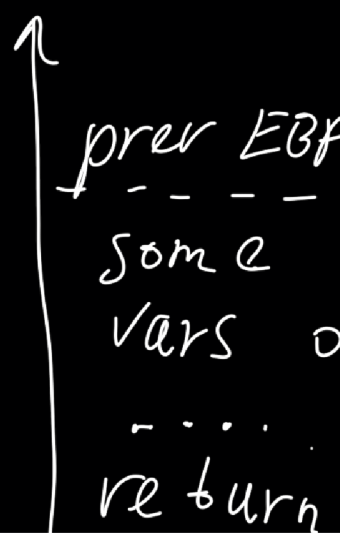
EBP – еще один регистр, указывает

• Такое устройство фрейма позволяет

не будут использовать нашу память.

- Пролог функции, в которой выделяется стек, называется прологом.
- Конец функции, в которой восстанавливается всё, что было изменено, называется эпилогом.

- Итак, давайте попробуем написать пролог и эпилог для функции provideSecret (из файла)
- На момент вызова нашей функции у нас следующее состояние стека:



- Поговорим о вызовах функций:

1) Каким образом мы можем передавать аргументы функции? [вопрос к аудитории]

- Через регистры
- Через стек

- Например, одна из наших библиотечных функций: io_print_hex

- Как мы узнали, как именно передавать в нее аргументы, где искать возвращаемое значение, какие регистры она портит, а какие - нет? [вопрос в аудиторию]

=> Посмотрели в документации.

[открыть доску]

- А если документации нет?
[вопрос в аудиторию]

- Кажется, неудобно. Для решения этой проблемы придумали соглашения о вызовах. (calling conventions)

Это набор правил, которые говорят нам:
- Кто очищает стек
- Как передаются аргументы
- Какие регистры caller-saved, какие callee-saved.

- В x86 существует множество соглашений о вызовах. Рассмотрим некоторые:

1) Cdecl: (самое распространённое соглашение в 32х битах

программы под Unix).
- Аргументы передаются через стек:

```
foo(a,b,c) →  push c
              push b
              push a
              call foo
```

- Очистку стека выполняет вызывающая сторона..
Т.е. после call foo, мы делаем add esp, 12 (3 поч)
- Возвращаемое значение в EAX, либо пара EDX:EAX, либо через память
- caller-saved: EAX, ECX, EDX
- callee-saved: EBX, ESI, EDI, EBP

И System V: (часто используется в 64-х битных системах)
- Первые 6 аргументов через RDI, RSI, RDX, RCX, R8, R9, остальные - через стек

- Стек выделяет и очищает сам
- Возвращаемое значение – аналогично cdecl
- Caller-saved: RAX, RCX, RDX, RSI, RDI, R8–R11
- Callee-saved: RBX, RBP, R12–R15

- Как работают удобные функции:

ID_ATTR – специальный макрос, который говорит, что нужно использовать регистры для передачи аргументов.